

Name: Vũ Đại Dương

ID: 23520359

Class: IT007.P28.2

OPERATING SYSTEM LAB 4'S REPORT

SUMMARY

Task		Status	Page
Section 4.5	Exs 1	Done	2 - 6
	Exs 2	Done	7 - 11
...	...		
	...		

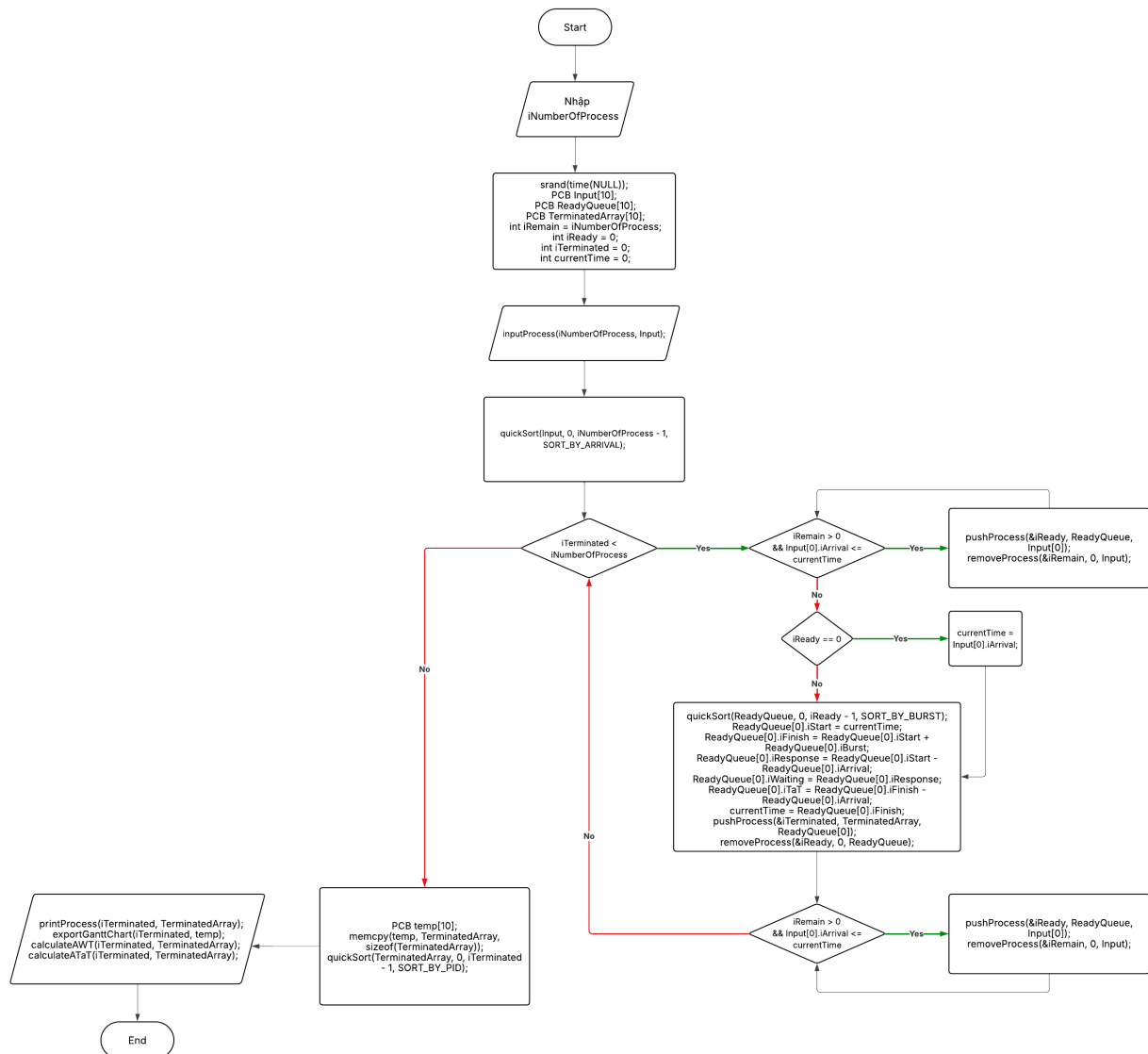
Self-scores:

Note: Export file to **PDF and name the file by following format:
Student ID_LABx.pdf*

Section 4.5

1. Viết chương trình mô phỏng giải thuật SJF

1.1: Lưu đồ giải thuật:



- Giải thích:

- **B1:** Nhập số tiến trình
- **B2:**
 - Srand(time(NULL)): hàm random
 - Khởi tạo các array: Input (các tiến trình đã nhập), ReadyQueue (các tiến trình đang chạy), Terminated (các tiến trình đã hoàn thành) và các biến số lượng phần tử của từng array (iReady, iTerminated)

- Khởi tạo biến iRemain (đếm các tiến trình còn lại), currenttime (thời gian chạy)
- **B3:** Nhập dữ liệu của từng tiến trình (PID, Arrival Time, Burst Time)
- **B4:** Sắp xếp các tiến trình theo trình tự Arrival Time (sử dụng quickSort)
- **B5:** Cho các tiến trình vào vòng lặp lấy ra phần tử ngoài cùng lúc này tiến trình đầu tiên được thực thi, currenttime đã được cộng thêm bursttime của tiến trình đó, lúc này currenttime là thời gian kết thúc của tiến trình trong vòng lặp.
- **B6:** Tính toán iStart, iFinish, iResponse, iWaiting, iTaT
- **B7:** Sắp xếp các tiến trình còn lại theo Burst Time - quickSort(ReadyQueue, 0, iReady - 1, SORT_BY_BURST); và lặp lại bước 5
- **B8:** Sau khi hoàn thành, in ra dữ liệu của từng tiến trình theo trình tự PID, in ra Gantt Chart và AWT, AtaT.

1.2: Code giải thuật

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX 100
#define SORT_BY_ARRIVAL 0
#define SORT_BY_PID 1
#define SORT_BY_BURST 2
#define SORT_BY_START 3

typedef struct {
    int iPID;
    int iArrival, iBurst;
    int iStart, iFinish, iWaiting, iResponse, iTaT;
} PCB;

void inputProcess(int n, PCB P[]) {
    for (int i = 0; i < n; i++) {
        P[i].iPID = i + 1;

        //Random Arrival Time
        P[i].iArrival = rand() % 21;    // [0, 20]

        //Random Burst Time
        P[i].iBurst = (rand() % 11) + 2;    // [2, 12]

        printf("Process ID, Arrival Time, Burst Time: %d %d %d\n", P[i].iPID, P[i].iArrival, P[i].iBurst);

        P[i].iStart = 0;
        P[i].iFinish = 0;
        P[i].iWaiting = 0;
        P[i].iResponse = 0;
        P[i].iTaT = 0;
    }
}
```

```

void printProcess(int n, PCB P[]){
    for (int i = 0; i < n; i++){
        {
            printf("PID: %d, Arrival: %d, Burst: %d, Start: %d, Finish: %d, Waiting: %d, Response: %d, TaT: %d\n",
                P[i].iPID, P[i].iArrival, P[i].iBurst,
                P[i].iStart, P[i].iFinish,
                P[i].iWaiting, P[i].iResponse,
                P[i].iTAT);
        }
    }
}

void exportGanttChart (int n, PCB P[]){
    printf("\nGantt Chart:\n");
    for (int i = 0; i < n; i++) {
        printf(" P%d |", P[i].iPID);
    }
    printf("\n%d", P[0].iStart);
    for (int i = 0; i < n; i++) {
        printf("    %d", P[i].iFinish);
    }
    printf("\n");
}

void pushProcess(int *n, PCB P[], PCB Q) {
    P[*n] = Q;
    (*n)++;
}

void removeProcess(int *n, int index, PCB P[]){
    for (int i = index; i < *n - 1; i++){
        {
            P[i] = P[i + 1];
        }
    }
    (*n)--;
}

int swapProcess(PCB *P, PCB *Q) {
    PCB temp = *P;
    *P = *Q;
    *Q = temp;
}

int partition (PCB P[], int low, int high, int iCriteria){
    PCB pivot = P[high];
    int i = (low - 1);
    for (int j = low; j < high; j++)
    {
        if ((iCriteria == SORT_BY_ARRIVAL && P[j].iArrival <= pivot.iArrival) ||
            (iCriteria == SORT_BY_PID && P[j].iPID <= pivot.iPID) ||
            (iCriteria == SORT_BY_BURST && P[j].iBurst <= pivot.iBurst) ||
            (iCriteria == SORT_BY_START && P[j].iStart <= pivot.iStart))
        {
            i++;
            swapProcess(&P[i], &P[j]);
        }
    }
    swapProcess(&P[i + 1], &P[high]);
    return (i + 1);
}

void quickSort(PCB P[], int low, int high, int iCriteria) {
    if (low < high)
    {
        int pi = partition(P, low, high, iCriteria);
        quickSort(P, low, pi - 1, iCriteria);
        quickSort(P, pi + 1, high, iCriteria);
    }
}

```

```

void calculateAWT(int n, PCB P[]) {
    float totalWT = 0;
    for (int i = 0; i < n; i++)
    {
        totalWT += P[i].iWaiting;
    }
    printf("\nAWT: %.2f", totalWT / n);
}

void calculateATaT(int n, PCB P[]) {
    float totalTaT = 0;
    for (int i = 0; i < n; i++)
    {
        totalTaT += P[i].iTaT;
    }
    printf("\nATA T: %.2f\n", totalTaT / n);
}

int main()
{
    srand(time(NULL));

    PCB Input[10];
    PCB ReadyQueue[10];
    PCB TerminatedArray[10];

    int iNumberOfProcess;
    printf("Please input number of Process: ");
    scanf("%d", &iNumberOfProcess);

    int iRemain = iNumberOfProcess, iReady = 0, iTerminated = 0;
    int currentTime = 0;

    inputProcess(iNumberOfProcess, Input);
    quickSort(Input, 0, iNumberOfProcess - 1, SORT_BY_ARRIVAL);

    while (iTerminated < iNumberOfProcess)
    {
        while (iRemain > 0 && Input[0].iArrival <= currentTime)
        {
            pushProcess(&iReady, ReadyQueue, Input[0]);
            removeProcess(&iRemain, 0, Input);
        }

        if (iReady == 0)
        {
            currentTime = Input[0].iArrival;
            continue;
        }

        quickSort(ReadyQueue, 0, iReady - 1, SORT_BY_BURST);

        ReadyQueue[0].iStart = currentTime;
        ReadyQueue[0].iFinish = ReadyQueue[0].iStart + ReadyQueue[0].iBurst;
        ReadyQueue[0].iResponse = ReadyQueue[0].iStart - ReadyQueue[0].iArrival;
        ReadyQueue[0].iWaiting = ReadyQueue[0].iResponse;
        ReadyQueue[0].iTaT = ReadyQueue[0].iFinish - ReadyQueue[0].iArrival;

        currentTime = ReadyQueue[0].iFinish;

        pushProcess(&iTerminated, TerminatedArray, ReadyQueue[0]);
        removeProcess(&iReady, 0, ReadyQueue);

        while (iRemain > 0 && Input[0].iArrival <= currentTime)
        {
            pushProcess(&iReady, ReadyQueue, Input[0]);
            removeProcess(&iRemain, 0, Input);
        }
    }

    printf("\n===== SJF Scheduling =====\n");

    PCB temp[10];
    memcpy(temp, TerminatedArray, sizeof(TerminatedArray));
}

```

```

quickSort(TerminatedArray, 0, iTerminated - 1, SORT_BY_PID);
printProcess(iTerminated, TerminatedArray);

exportGanttChart(iTerminated, temp);

calculateAWT(iTerminated, TerminatedArray);
calculateATaT(iTerminated, TerminatedArray);

return 0;
}

```

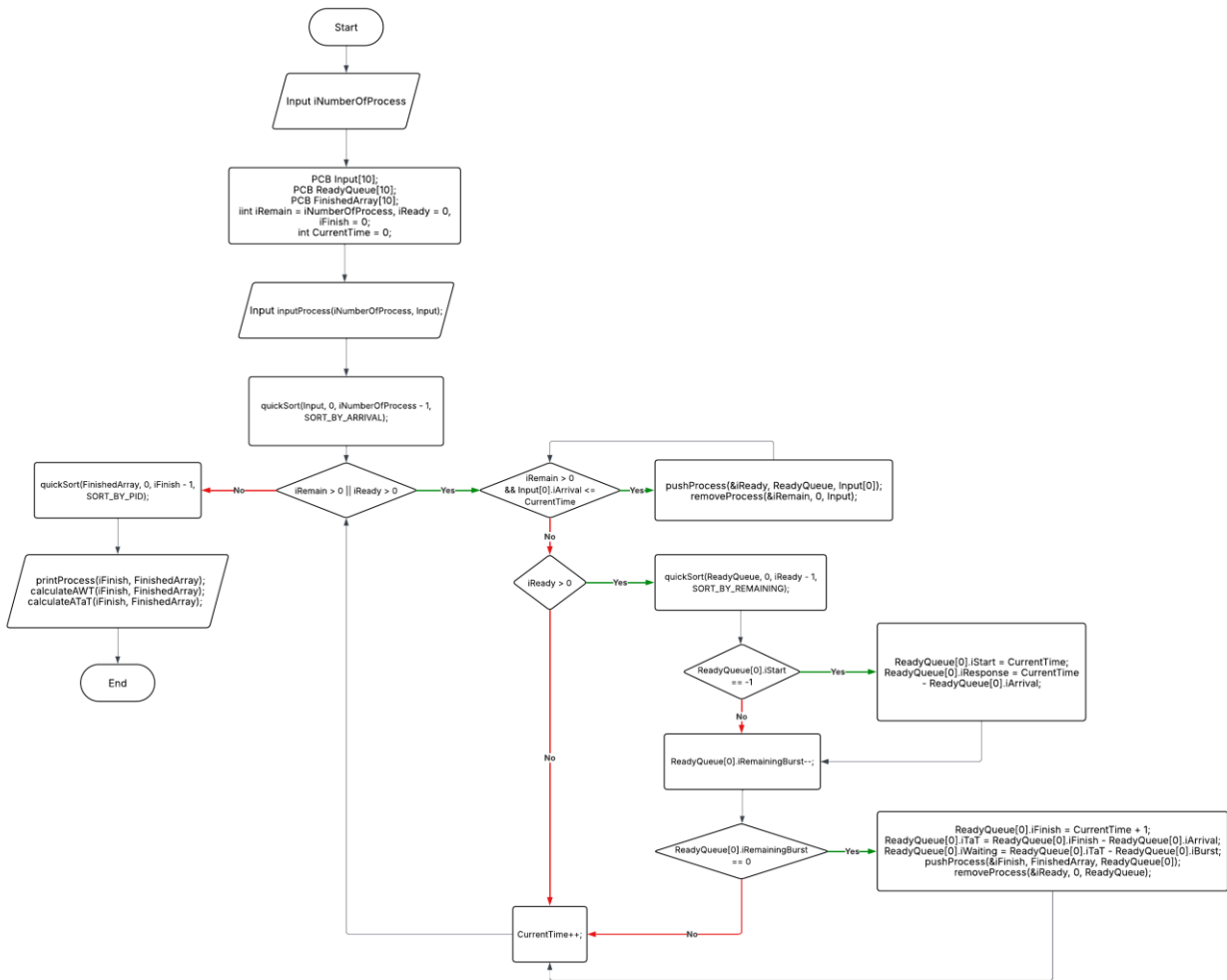
1.3: Testcase

	Chạy code	Chạy tay
Testcase 1	<pre> Please input number of Process: 5 Process ID, Arrival Time, Burst Time: 1 9 9 Process ID, Arrival Time, Burst Time: 2 4 10 Process ID, Arrival Time, Burst Time: 3 19 3 Process ID, Arrival Time, Burst Time: 4 19 2 Process ID, Arrival Time, Burst Time: 5 13 12 ===== SJF Scheduling ===== PID: 1, Arrival: 9, Burst: 9, Start: 14, Finish: 23, Waiting: 5, Response: 5, TaT: 14 PID: 2, Arrival: 4, Burst: 10, Start: 4, Finish: 14, Waiting: 0, Response: 0, TaT: 10 PID: 3, Arrival: 19, Burst: 3, Start: 25, Finish: 28, Waiting: 6, Response: 6, TaT: 9 PID: 4, Arrival: 19, Burst: 2, Start: 23, Finish: 25, Waiting: 4, Response: 4, TaT: 6 PID: 5, Arrival: 13, Burst: 12, Start: 28, Finish: 40, Waiting: 15, Response: 15, TaT: 27 Gantt Chart: P2 P1 P4 P3 P5 4 14 23 25 28 40 AWT: 6.00 ATaT: 13.20 </pre>	Handwritten test case 1 shows a table of processes with their IDs, arrival times, and burst times. Below the table is a Gantt chart showing the execution order of processes P2, P1, P4, P3, and P5. The Gantt chart includes time markers at 4, 14, 23, 25, 28, and 40. Calculations for AWT (6.00) and ATaT (13.20) are shown at the bottom.
Testcase 2	<pre> Please input number of Process: 5 Process ID, Arrival Time, Burst Time: 1 13 2 Process ID, Arrival Time, Burst Time: 2 4 7 Process ID, Arrival Time, Burst Time: 3 17 3 Process ID, Arrival Time, Burst Time: 4 16 7 Process ID, Arrival Time, Burst Time: 5 9 5 ===== SJF Scheduling ===== PID: 1, Arrival: 13, Burst: 2, Start: 16, Finish: 18, Waiting: 3, Response: 3, TaT: 5 PID: 2, Arrival: 4, Burst: 7, Start: 4, Finish: 11, Waiting: 0, Response: 0, TaT: 7 PID: 3, Arrival: 17, Burst: 3, Start: 18, Finish: 21, Waiting: 1, Response: 1, TaT: 4 PID: 4, Arrival: 16, Burst: 7, Start: 21, Finish: 28, Waiting: 5, Response: 5, TaT: 12 PID: 5, Arrival: 9, Burst: 5, Start: 11, Finish: 16, Waiting: 2, Response: 2, TaT: 7 Gantt Chart: P2 P5 P1 P3 P4 4 11 16 18 21 28 AWT: 2.20 ATaT: 7.00 </pre>	Handwritten test case 2 shows a table of processes with their IDs, arrival times, and burst times. Below the table is a Gantt chart showing the execution order of processes P2, P5, P1, P3, and P4. The Gantt chart includes time markers at 4, 11, 16, 18, 21, and 28. Calculations for AWT (2.20) and ATaT (7.00) are shown at the bottom.
Testcase 3	<pre> Please input number of Process: 5 Process ID, Arrival Time, Burst Time: 1 11 11 Process ID, Arrival Time, Burst Time: 2 5 9 Process ID, Arrival Time, Burst Time: 3 17 4 Process ID, Arrival Time, Burst Time: 4 8 9 Process ID, Arrival Time, Burst Time: 5 0 11 ===== SJF Scheduling ===== PID: 1, Arrival: 11, Burst: 11, Start: 33, Finish: 44, Waiting: 22, Response: 22, TaT: 33 PID: 2, Arrival: 5, Burst: 9, Start: 11, Finish: 20, Waiting: 6, Response: 6, TaT: 15 PID: 3, Arrival: 17, Burst: 4, Start: 20, Finish: 24, Waiting: 3, Response: 3, TaT: 7 PID: 4, Arrival: 8, Burst: 9, Start: 24, Finish: 33, Waiting: 16, Response: 16, TaT: 25 PID: 5, Arrival: 0, Burst: 11, Start: 0, Finish: 11, Waiting: 0, Response: 0, TaT: 11 Gantt Chart: P5 P2 P3 P4 P1 0 11 20 24 33 44 AWT: 9.40 ATaT: 18.20 </pre>	Handwritten test case 3 shows a table of processes with their IDs, arrival times, and burst times. Below the table is a Gantt chart showing the execution order of processes P5, P2, P3, P4, and P1. The Gantt chart includes time markers at 0, 11, 20, 24, 33, and 44. Calculations for AWT (9.40) and ATaT (18.20) are shown at the bottom.

- Nhận xét: Kết quả chạy tay và chạy code hoàn toàn giống nhau

2. Viết chương trình giải thuật SRTF

1.1: Lưu đồ giải thuật



- Giải thích:

- **B1:** Nhập số lượng tiến trình (iNumberOfProcess)
- **B2:** Khởi tạo các biến
 - Srand(time(NULL)): hàm random
 - Khởi tạo các array: Input (các tiến trình đã nhập), ReadyQueue (các tiến trình đang chạy), Terminated (các tiến trình đã hoàn thành) và các biến số lượng phần tử của từng array (iReady, iFinish)
 - Khởi tạo biến iRemain (đếm các tiến trình còn lại), currenttime (thời gian chạy)
- **B3:** Nhập dữ liệu cần thiết cho từng tiến trình (PID, Arrival Time, Burst Time)
- **B4:** Sắp xếp thứ tự tiến trình theo Arrival Time (sử dụng quickSort)
- **B5:** Đưa tiến trình vào ReadyQueue, xóa khỏi Input

- **B6:** Nếu có tiến trình trong ReadyQueue, sắp xếp theo Burst Time → Tiến trình có Burst Time nhỏ nhất luôn nằm ở ReadyQueue[0].
- **B7:** Nếu tiến trình lần đầu tiên được cấp CPU, ghi lại Start Time và Response Time của tiến trình
- **B8:** Giảm Burst Time
- **B9:** Nếu tiến trình hoàn thành, tính toán các giá trị: Finish Time, TaT, Waiting Time → chuyển tiến trình từ ReadyQueue qua FinishedArray → xóa tiến trình ở ReadyQueue
- **B10:** Tăng currentTime, lặp lại B5 cho đến khi tất cả tiến trình hoàn thành
- **B11:** Sau khi hoàn thành, tính toán ART, AtaT, AWT

1.2: Code giải thuật

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SORT_BY_ARRIVAL 0
#define SORT_BY_PID 1
#define SORT_BY_BURST 2
#define SORT_BY_START 3
#define SORT_BY_REMAINING 4

typedef struct
{
    int iPID;
    int iArrival, iBurst;
    int iStart, iFinish, iWaiting, iResponse, iTaT, iRemainingBurst;
} PCB;

void inputProcess(int n, PCB P[]) {
    for (int i = 0; i < n; i++) {
        P[i].iPID = i + 1;
        P[i].iArrival = rand() % 21; // Random arrival time [0, 20]
        P[i].iBurst = (rand() % 11) + 2; // Random burst time [2, 12]
        P[i].iRemainingBurst = P[i].iBurst;
        printf("Process ID: %d, Arrival Time: %d, Burst Time: %d\n",
            P[i].iPID, P[i].iArrival, P[i].iBurst);
        P[i].iStart = -1;
        P[i].iFinish = -1;
        P[i].iWaiting = 0;
        P[i].iResponse = -1;
        P[i].iTaT = 0;
    }
}
```



```

void printProcess(int n, PCB P[]) {
    for (int i = 0; i < n; i++) {
        printf("PID: %d, Arrival: %d, Burst: %d, Start: %d, Finish: %d, Waiting: %d, Response: %d, TaT: %d\n",
            P[i].iPID, P[i].iArrival, P[i].iBurst,
            P[i].iStart, P[i].iFinish,
            P[i].iWaiting, P[i].iResponse,
            P[i].iTaT);
    }
}

void pushProcess(int *n, PCB P[], PCB Q) {
    P[*n] = Q;
    (*n)++;
}

void removeProcess(int *n, int index, PCB P[]) {
    if ((*n) <= 0 || index < 0 || index >= (*n)) {
        return;
    }
    for (int i = index; i < (*n) - 1; i++) {
        P[i] = P[i + 1];
    }
    (*n)--;
}

int swapProcess(PCB *P, PCB *Q) {
    PCB temp = *P;
    *P = *Q;
    *Q = temp;
    return 1;
}

int partition(PCB P[], int low, int high, int iCriteria) {
    int i = (low - 1);
    for (int j = low; j < high; j++) {
        if ((iCriteria == SORT_BY_ARRIVAL && P[j].iArrival <= pivot.iArrival) ||
            (iCriteria == SORT_BY_PID && P[j].iPID <= pivot.iPID) ||
            (iCriteria == SORT_BY_BURST && P[j].iBurst <= pivot.iBurst) ||
            (iCriteria == SORT_BY_START && P[j].iStart <= pivot.iStart) ||
            (iCriteria == SORT_BY_REMAINING && P[j].iRemainingBurst <= pivot.iRemainingBurst)) {
            i++;
            swapProcess(&P[i], &P[j]);
        }
    }
    swapProcess(&P[i + 1], &P[high]);
    return (i + 1);
}

void quickSort(PCB P[], int low, int high, int iCriteria) {
    if (low < high) {
        int pi = partition(P, low, high, iCriteria);
        quickSort(P, low, pi - 1, iCriteria);
        quickSort(P, pi + 1, high, iCriteria);
    }
}

void calculateART(int n, PCB P[]) {
    double sum = 0;
    for (int i = 0; i < n; i++) {
        sum += P[i].iResponse;
    }
    printf("\nAverage Response Time: %.2f\n", sum/n);
}

void calculateAWT(int n, PCB P[]) {
    double sum = 0;
    for (int i = 0; i < n; i++) {
        sum += P[i].iWaiting;
    }
    printf("Average Waiting Time: %.2f\n", sum/n);
}

```

```

void calculateATaT(int n, PCB P[]) {
    for(int i = 0; i < n; i++) {
        sum += P[i].iTaT;
    }
    printf("Average Turnaround Time: %.2f\n", sum/n);
}

void printGanttChart(int GanttProcess[], int GanttTime[], int ganttIndex) {
    printf("\nGantt Chart:\n");
    for (int i = 0; i < ganttIndex; i++) {
        printf("| P%d ", GanttProcess[i]);
    }
    printf("\n");
    for (int i = 0; i <= ganttIndex; i++) {
        printf("    ", GanttTime[i]);
    }
    printf("\n");
}

int main() {
    srand(time(NULL));

    PCB Input[10];
    PCB ReadyQueue[10];
    PCB FinishedArray[10];
    int GanttProcess[100], GanttTime[100];
    int ganttIndex = 0;

    int iNumberOfProcess;
    printf("Please input number of Process: ");
    scanf("%d", &iNumberOfProcess);

    int iRemain = iNumberOfProcess, iReady = 0, iFinish = 0;
    int CurrentTime = 0;

    inputProcess(iNumberOfProcess, Input);
    quickSort(Input, 0, iNumberOfProcess - 1, SORT_BY_ARRIVAL);

    while (iRemain > 0 || iReady > 0) {
        // Move arrived processes into ReadyQueue
        while (iRemain > 0 && Input[0].iArrival <= CurrentTime) {
            pushProcess(&iReady, ReadyQueue, Input[0]);
            removeProcess(&iRemain, 0, Input);
        }

        // If ReadyQueue not empty, process
        if (iReady > 0) {
            quickSort(ReadyQueue, 0, iReady - 1, SORT_BY_REMAINING);

            // If this is a new process (or preemption), record in Gantt chart
            if (ganttIndex == 0 || ReadyQueue[0].iPID != GanttProcess[ganttIndex - 1]) {
                GanttProcess[ganttIndex] = ReadyQueue[0].iPID;
                GanttTime[ganttIndex] = CurrentTime;
                ganttIndex++;
            }

            // If first time running, set Start
            if (ReadyQueue[0].iStart == -1) {
                ReadyQueue[0].iStart = CurrentTime;
                ReadyQueue[0].iResponse = CurrentTime - ReadyQueue[0].iArrival;
            }

            ReadyQueue[0].iRemainingBurst--;

            // If finished
            if (ReadyQueue[0].iRemainingBurst == 0) {
                ReadyQueue[0].iFinish = CurrentTime + 1;
                ReadyQueue[0].iTaT = ReadyQueue[0].iFinish - ReadyQueue[0].iArrival;
                ReadyQueue[0].iWaiting = ReadyQueue[0].iTaT - ReadyQueue[0].iBurst;
                pushProcess(&iFinish, FinishedArray, ReadyQueue[0]);
                removeProcess(&iReady, 0, ReadyQueue);
            }
            CurrentTime++;
        }
    }
}

```

```

    }
    else { // No process ready, CPU idle
        CurrentTime++;
    }
}

// Add final Gantt time
GanttTime[ganttIndex] = CurrentTime;

printf("\n==== SRTF Scheduling =====\n");
quickSort(FinishedArray, 0, iFinish - 1, SORT_BY_PID);
printProcess(iFinish, FinishedArray);

printGanttChart(GanttProcess, GanttTime, ganttIndex);
calculateART(iFinish, FinishedArray);
calculateAWT(iFinish, FinishedArray);
calculateATAT(iFinish, FinishedArray);

return 0;
}

```

1.3: Testcase

	Chạy code	Chạy tay
Testcase 1	Please input number of Process: 5 Process ID: 1, Arrival Time: 20, Burst Time: 6 Process ID: 2, Arrival Time: 14, Burst Time: 2 Process ID: 3, Arrival Time: 7, Burst Time: 10 Process ID: 4, Arrival Time: 6, Burst Time: 4 Process ID: 5, Arrival Time: 8, Burst Time: 2 ==== SRTF Scheduling ===== PID: 1, Arrival: 20, Burst: 6, Start: 24, Finish: 30, Waiting: 4, Response: 4, TaT: 10 PID: 2, Arrival: 14, Burst: 2, Start: 14, Finish: 16, Waiting: 0, Response: 0, TaT: 2 PID: 3, Arrival: 7, Burst: 10, Start: 12, Finish: 24, Waiting: 7, Response: 5, TaT: 17 PID: 4, Arrival: 6, Burst: 4, Start: 6, Finish: 10, Waiting: 0, Response: 0, TaT: 4 PID: 5, Arrival: 8, Burst: 2, Start: 10, Finish: 12, Waiting: 2, Response: 2, TaT: 4 Gantt Chart: <pre> P4 P5 P3 P2 P3 P1 6 10 12 14 16 24 30 </pre> Average Response Time: 2.20 Average Waiting Time: 2.60 Average Turnaround Time: 7.40	
Testcase 2	Please input number of Process: 5 Process ID: 1, Arrival Time: 8, Burst Time: 10 Process ID: 2, Arrival Time: 20, Burst Time: 10 Process ID: 3, Arrival Time: 13, Burst Time: 3 Process ID: 4, Arrival Time: 17, Burst Time: 2 Process ID: 5, Arrival Time: 18, Burst Time: 12 ==== SRTF Scheduling ===== PID: 1, Arrival: 8, Burst: 10, Start: 8, Finish: 23, Waiting: 5, Response: 0, TaT: 15 PID: 2, Arrival: 20, Burst: 10, Start: 23, Finish: 33, Waiting: 3, Response: 3, TaT: 13 PID: 3, Arrival: 13, Burst: 3, Start: 13, Finish: 16, Waiting: 0, Response: 0, TaT: 3 PID: 4, Arrival: 17, Burst: 2, Start: 17, Finish: 19, Waiting: 0, Response: 0, TaT: 2 PID: 5, Arrival: 18, Burst: 12, Start: 33, Finish: 45, Waiting: 15, Response: 15, TaT: 27 Gantt Chart: <pre> P1 P3 P1 P4 P1 P2 P5 8 13 16 17 19 23 33 45 </pre> Average Response Time: 3.60 Average Waiting Time: 4.60 Average Turnaround Time: 12.00	
Testcase 3	Please input number of Process: 5 Process ID: 1, Arrival Time: 3, Burst Time: 8 Process ID: 2, Arrival Time: 8, Burst Time: 4 Process ID: 3, Arrival Time: 11, Burst Time: 9 Process ID: 4, Arrival Time: 15, Burst Time: 6 Process ID: 5, Arrival Time: 16, Burst Time: 4 ==== SRTF Scheduling ===== PID: 1, Arrival: 3, Burst: 8, Start: 3, Finish: 11, Waiting: 0, Response: 0, TaT: 8 PID: 2, Arrival: 8, Burst: 4, Start: 11, Finish: 15, Waiting: 3, Response: 3, TaT: 7 PID: 3, Arrival: 11, Burst: 9, Start: 25, Finish: 34, Waiting: 14, Response: 14, TaT: 23 PID: 4, Arrival: 15, Burst: 6, Start: 15, Finish: 25, Waiting: 4, Response: 0, TaT: 10 PID: 5, Arrival: 16, Burst: 4, Start: 16, Finish: 20, Waiting: 0, Response: 0, TaT: 4 Gantt Chart: <pre> P1 P2 P4 P5 P4 P3 3 11 15 16 20 25 34 </pre> Average Response Time: 3.40 Average Waiting Time: 4.20 Average Turnaround Time: 10.40	

- Nhận xét: Kết quả chạy code và chạy tay hoàn toàn giống nhau.